Data fetching patterns

Module 4 - Lesson 3

Overview

Where and how you fetch data changes your app’s performance and correctness. Server Components fetch during render, while caching and streaming keep results fast and interactive.

Key Concepts

- Fetch in Server Components to read databases and APIs during server render.
- Use fetch with cache options or a data library like TanStack Query for client data.
- Parallel requests with Promise.all reduce total latency; avoid waterfall fetches.
- Streaming with loading.tsx shows UI immediately while slower parts finish.
- Cache data that rarely changes; revalidate periodically or on mutation.
- Handle empty, loading, and error states explicitly in the UI.

Example

```
// Server Component - no client fetch needed
async function Dashboard() {
  const [courses, stats] = await Promise.all([
    db.query.courses.findMany(),
    db.query.enrollments.count(),
  ]);
  return <DashboardView courses={courses} stats={stats} />;
}
```

Summary

Data fetching belongs as close to the server as possible. Server Components, parallel requests, caching, and streaming give fast pages with explicit loading and error states.

Practice Checklist

- Fetch two resources in parallel with Promise.all in a Server Component.
- Add a loading.tsx file and observe the streaming behavior.
- Set a cache/revalidation policy on a slow endpoint.